

Provable Secure Anonymous Device Authentication Protocol in IoT Environment

Shanyao Ren¹, Yizhong Liu¹, *Member, IEEE*, Beiyuan Yu, Jianwei Liu², *Senior Member, IEEE*, and Dongyu Li³, *Member, IEEE*

Abstract—The inherent massive heterogeneous devices and open channels in the Internet of Things (IoT) present significant challenges for identity authentication between devices and cloud servers. For this issue, reliable protocols ensure the legality of participants and act as a crucial method to provide security for authentication. In previous research, schemes devised by researchers exhibit certain security vulnerabilities, making it challenging to withstand comprehensive network attacks, e.g., stolen device attacks, replay attacks, impersonation, etc. Additionally, some protocols have complex interaction processes, which incur significant computational redundancy and resource loss. Motivated by this, this article proposes an anonymous and certificateless lightweight authentication protocol (ACLAP) for device-to-server and device-to-device based on elliptic curve cryptography. It improves the communication quality between devices and cloud servers and solves the security risks in authentication. In the scheme, we utilize device users' passwords and biometric features as verification credentials without storing any trusted proofs on the cloud server. We address the issue of resource consumption caused by numerous devices in the IoT environment. From formal security analysis and comparisons with other works, our protocol has preferable security performance and effectively saves communication resources for authentication. Simulation results demonstrate the feasibility and practical significance of the scheme.

Index Terms—Authentication scheme, Internet of Things (IoT) devices, real-or-random (ROR) model.

I. INTRODUCTION

THE Internet of Things (IoT) is a revolutionary communication system that enables interconnectivity among a diverse range of objects, such as sensors, vehicles, and smart devices [1], [2]. In the IoT, there are numerous terminal

Manuscript received 25 July 2023; revised 8 October 2023; accepted 9 November 2023. Date of publication 15 November 2023; date of current version 26 March 2024. This work was supported in part by the National Key R&D Program of China under Grant 2021YFB2700200; in part by the National Natural Science Foundation of China under Grant U21B2021, Grant 61972018, and Grant 61932014; in part by the Industry-University-Research Foundation of China under Grant 2021ZYA02022; in part by the Foundation of Science and Technology on Space Intelligent Control Laboratory under Grant HTKJ2022KL502008; and in part by the Tianmushan Laboratory under Grant TK-2023-B-010 and Grant TK-2023-C-020. (Corresponding author: Dongyu Li.)

Shanyao Ren and Jianwei Liu are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China, and also with the Hangzhou Innovation Institute, Beihang University, Hangzhou 310051, China.

Yizhong Liu and Beiyuan Yu are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China.

Dongyu Li is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, also with Tianmushan Laboratory, Hangzhou 310023, China, and also with the Shanghai Institute of Satellite Engineering, Shanghai 201109, China (e-mail: dongyuli@buaa.edu.cn).

Digital Object Identifier 10.1109/IIOT.2023.3332943

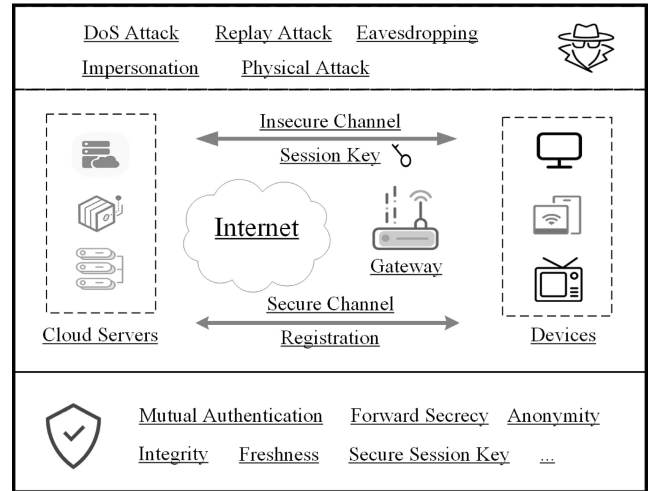


Fig. 1. Device authentication network in the IoT.

devices that are difficult to supervise physically, making security management weaker. Similarly, communication between devices is susceptible to attacks [3], such as impersonation, Denial-of-Service (DoS), replay attacks, etc.

Devices connect to servers through the Internet, register in a secure environment, and communicate through session keys over an insecure channel. Due to the lack of identity verification among terminal devices, attackers can exploit this weakness and launch continuous intrusion on both parties, as shown in Fig. 1. Therefore, addressing authentication and communication security issues is crucial for achieving IoT security [4]. The utilization of lightweight protocols is deemed an effective approach to realize the authentication of resource-constrained scenarios, which guarantees confidentiality, integrity, and nonrepudiation of the communication. These protocols typically employ lightweight security techniques, such as symmetric encryption, public-key cryptosystem, and hash functions [5], [6].

There are several unique characteristics to consider for protocols in various scenarios of the IoT, such as massive devices, limited node energy, diverse application requirements, and so on [7]. Privacy-protected protocols may fail in resisting comprehensive attacks without compromising efficiency and security. In addition, storing a large amount of information on the server side may cause resource congestion and confront the risk of data leakage. Due to these circumstances, the design of a secure protocol faces two main obstacles [8].

- 1) *Security Requirements*: Schemes require detailed interaction processes and ensure that communication parties can complete authentication without compromising the confidentiality of information. They should undergo security analysis, such as BAN logic or formal security proofs, to ensure their effectiveness.
- 2) *Efficiency*: Complex designs undertaken to ensure security inevitably increase the number of interactions and the volume of transmitted information between devices and servers. They lead to higher bandwidth consumption and latency.

In practical IoT environments, striking a delicate equilibrium between security and efficiency is challenging for authentication protocols due to the inherent tradeoff between these two aspects. Our objective is to attain this balance in a resource-constrained IoT environment.

In this work, we propose an anonymous, certificateless, and lightweight authentication protocol (ACLAP) for the IoT environment. The ACLAP simultaneously achieves security authentication and session key sharing between the device and the cloud server (D2C), as well as between the device and the device (D2D). It processes interactive information anonymously, eliminating the need for server-based storage of private data. Consequently, it effectively mitigates the risk of stolen verifier attacks and information leakage. By rigorous analysis, the ACLAP successfully provides forward security and resists common Internet attacks, including DoS attacks, replay attacks, impersonation attacks, and other associated vulnerabilities. The significance of our protocol lies in its potential applications: 1) it can be applied in situations where there are massive devices to communicate, thus reducing the storage costs for cloud servers and 2) it has preferable scalability for utilization in diverse IoT authentication scenarios, e.g., intelligent transportation, smart home, wearable healthcare, etc.

The remainder of this article is organized as follows. Section II provides recent works on the communication protocols for the IoT environment. In Section III, we present the cryptographic background knowledge utilized in our work and point out the flaws of previous schemes. The specific process of the ACLAP is proposed in detail in Section IV, including the registration and authentication phases among devices and the cloud server. In Section V, our scheme is proven secure through common attacks defense and formal security analysis. Based on theoretical results, we simulate the protocol in Section VI, in which we compare our work's security features and communication consumption with previous protocols. The conclusion is drawn in Section VII.

II. RELATED WORK AND CONTRIBUTIONS

A. Related Work

The authentication protocols in the IoT can be categorized based on their intended application domains, such as medical [9], [10], intelligent sensing [11], [12], vehicle networks [13], [14], and others. Additionally, they can also be classified based on the authentication methods and techniques they employ, such as physical unclonable functions (PUFs) [15], [16], [17], biometrics [18], blockchain [19],

fuzzy extractors [20], elliptic curve encryption algorithms [21], digital certificates [22], and others.

In the authentication process between devices, due to the different communication entities, protocols can be further classified into several categories. These include the D2C authentication [23], [24], [25], [26], [27], [28] and the D2D authentication [29], [30], [31], [32], [33], [34]. However, these security protocols suffer from security vulnerabilities, such as lack of anonymity in devices [23], [30], [31], [35], susceptibility to clock desynchronization attacks due to timestamps [25], [31], [32], vulnerability to verifier tables and device theft attacks [24], [26], [30], [32], and inability to provide forward secrecy [25]. In the IoT environment with massive devices, these flaws may cause serious risks, e.g., data leakage, integrity damage, identity authentication failure, etc. Moreover, some schemes exhibit excessive communication and storage overhead, resulting in network congestion and imposing a high load on devices.

Recently, He et al. [36] proposed a lightweight authentication and key exchange protocol with anonymity for IoT devices. Compared with other protocols, He's work exhibits advantages in terms of its lightweight nature and security measures. However, there are still points for improvement in their scheme. It lacks clock synchronization due to the utilization of timestamps and is vulnerable to threats, e.g., offline password guessing attacks, stolen device attacks, and device impersonation attacks. These issues can potentially lead to the unauthorized acquisition or manipulation of data during the authentication process, thereby compromising its effectiveness. They are challenging to address in practical environments due to the significant risks faced by communication participants in the IoT environment, including potential key disclosure, database leakage, etc. Additionally, attackers possess various intrusion methods and advanced techniques, infiltrating crucial authentication information from various angles. In our work, we overcome these security concerns by designing and improving our authentication protocol. We achieve this through several approaches, including avoiding the utilization of databases, employing more secure device authentication methods, and implementing multiple integrity checks. The detailed process is listed in Section IV.

B. Our Contributions

We analyze the contributions and limitations of state-of-the-art results in device authentication for the IoT environment.

- 1) We propose a lightweight, anonymous, untraceable, and certificateless communication scheme that can be applied to both device-to-server and device-to-device communication. Our protocol merely utilizes cryptographic calculations of hash functions and elliptic curves, eliminating the need for additional certificates or trusted credentials. It substantially reduces the burden on servers while ensuring security.
- 2) We prove the security of the proposed scheme by analyzing common attacks and employing the real-or-random (ROR) model. Through this analysis, we demonstrate that our scheme provides complete forward security and

effectively resists replay attacks, impersonation attacks, device stolen attacks, verifier table stolen attacks, offline password guessing attacks, and other potential threats.

- 3) We conduct a thorough security analysis of the device registration and authentication protocol proposed by He et al. [36]. Our review identifies several points for improvement in their protocol, including its vulnerability to attacks, such as device stolen attacks, offline password guessing attacks, and impersonation attacks. From an enhancement perspective, we change the encryption method used for protecting private information, ensuring that the communication process remains immune to these attacks. Additionally, we revise the communication overhead estimation method in [36], resulting in a more accurate and precise cost calculation.
- 4) Our proposed scheme is not limited to the current scenario but can be extended to cater to anonymous authentication requirements in the IoT. This enhanced scalability enables its application in diverse domains, including industrial automation, healthcare monitoring, smart transportation, etc.

III. PRELIMINARIES

A. Elliptic Curve Cryptography

The elliptic curve cryptography (ECC) is introduced in [37]. It applies shorter keys to achieve the same level of security as the RSA algorithm and is widely applied in encryption and authentication fields.

Here, we introduce the definition of notations in the ECC. Let F_p represent a finite field with the large prime number p . Based on this, an elliptic curve E is calculated as

$$y^2 = x^3 + ux + v \pmod{p} \quad (1)$$

where $u, v \in F_p$, and $4u^3 + 27v^2 \pmod{p} \neq 0$. With the addition and multiplication operation of the ECC, we have the definition of the elliptic curve discrete logarithm problem (ECDLP) as follows [38].

Definition 1: G is the finite cyclic group with the prime order p and generator g . Z_p^* is the multiplicative group with order $p - 1$. The ECDLP problem is to find a positive integer $x \in Z_p^*$ to realize $Q = x \cdot g$ with known Q and g , where Q is a random value on G .

The ECDLP problem is one of the most vital applicants of ECC. It is defined as difficulty over group G if attackers cannot obtain the value of x with a nonnegligible possibility within operation time. Unlike the general discrete logarithm problem in a finite multiplication group, the ECDLP over a finite field is more challenging to resolve, and the solution process is more complex. The ECDLP cannot be solved by all known algorithms in polynomial time currently.

Similar to the ECDLP, we introduce the definition of the elliptic curve Diffie–Hellman problem (ECDHP). The ECDHP key agreement algorithm is the integration of the ECC algorithm and the Diffie–Hellman key exchange principle, which is widely applied to the key agreement process of both parties. Parties can negotiate a key without sharing any secrets in the ECDHP.

Definition 2: M and D are points on G , with $M = m \cdot g$ and $D = d \cdot g$, $m, d \in Z_p^*$. Assume that $t = m \cdot d$, and $T = t \cdot g$.

The ECDHP aims to calculate T in the condition that values of M and D are known.

Remark 1: From the perspective of balancing computational efficiency, ECC has certain limitations. Lengthy keys result in additional storage overhead, and the intricate encryption and decryption processes lead to lower computational efficiency in ECC-based schemes. In resource-constrained environments, its performance might be surpassed by other security techniques, such as Hash-XOR-PUF-based solutions. Nevertheless, ECC offers high security and stability, making it suitable for scenarios with substantial security demands, especially when PUF-based schemes are vulnerable to modeling attacks and cannot guarantee the privacy of session keys. In this article, we utilize ECC for secure registration and authentication, while in practical environments, the choice of security technology should be based on a balance between resource and security requirements. Our aim is to reduce communication overhead and maintain security on the premise that the device hardware resources support ECC encryption and decryption operations. We plan to explore solutions incorporating a diverse range of technologies in our future work.

B. Security Requirements

To ensure that both parties of devices and the cloud server can complete the registration and authentication safely and effectively, the proposed scheme is required to meet the following security requirements as follows.

- 1) *Mutual Authentication:* Mutual authentication requires the exchange of session keys only after each party has authenticated the legitimacy of the other's identity, preventing attackers from impersonating either party.
- 2) *Perfect Forward Security:* Perfect forward security is achieved in the authentication process by making each session independent of the others. In case an attacker gains access to the master key and secret values of the cloud server, he cannot compute the previous session keys from them.
- 3) *Device Anonymity and Untraceability:* The protocol is designed to provide an anonymous access authentication mechanism to prevent unauthorized access to sensitive information, such as devices' identities.
- 4) *Defend Against Replay Attacks:* Open channels increase the likelihood of eavesdropping by attackers, who may also attempt to launch replay attacks by resending messages to devices or the cloud server. Hence, the proposed protocol incorporates an efficient anti-replay mechanism to ensure the messages' freshness is verified.
- 5) *Defend Against DoS Attacks:* In DoS attacks, attackers flood the network with a large number of messages containing authentication failures, which can occupy significant network resources. To prevent such attacks and conserve limited network bandwidth, our proposed scheme aims to provide fast and cost-effective authentication for legitimate users while rejecting illegal

requests, thereby reducing resource usage and mitigating the risk of DoS attacks.

We articulate other security requirements in more detail through informal security analysis in Section V.

C. Adversary Model

In this work, we employ the Dolev–Yao model to simulate the capabilities of attackers and analyze the protocol’s security. Dolev and Yao [39] jointly proposed a quantitative threat model, which reasonably limits the abilities of attackers and has been widely adopted as a standard threat model in the analysis of authentication protocols. In the Dolev–Yao model, attackers have the following three capabilities.

- 1) The attacker possesses the ability to eavesdrop on the entire network’s sessions and obtain any message that passes through the insecure channel. This implies that the attacker can acquire information from public channels utilized during the authentication phase between devices and the cloud server.
- 2) The attacker belongs to a legitimate communication entity in the network, granting him to initiate sessions with any communication entity and act as the intended recipient of messages from any entity.
- 3) The attacker possesses the capability to manipulate messages within the channel. This includes the ability to modify, forge, replicate, delete, intercept, and replay messages. Moreover, the attacker can impersonate any communication entity to transmit messages to others.

In the interaction of authentication protocol, it is assumed that all messages sent are susceptible to interception by the attacker, and all received messages are subject to processing by the attacker. However, the attacker is not all-powerful. The attacker lacks the following abilities.

- 1) The attacker is unable to guess the random number selected by the communication entity from a sufficiently large space.
- 2) The attacker cannot decrypt the intercepted ciphertext without a valid key, nor can they construct the corresponding ciphertext without a valid key.
- 3) The attacker lacks the ability to compute the corresponding private key based on the public key of a communication entity.

Based on the quantification of attacker capabilities mentioned above, the attacks that can be launched by attackers are listed as follows.

- 1) *Eavesdropping Attack*: The attacker intercepts messages transmitted through public channels via eavesdropping.
- 2) *Man-in-The-Middle Attack*: The attacker assumes the role of a relay between the communicating parties, intercepts messages in the channel, and establishes authentication separately with the embedded device and the cloud server. This allows the attacker to eavesdrop on or tamper with the exchanged messages between the two parties.
- 3) *Entity Impersonation*: The attacker constructs a pseudo user to access credential or authentication response

value. He impersonates the legitimate principal of the protocol to participate in the protocol operation.

- 4) *Replay Attack*: The attacker exploits the historical messages sent by the communication parties on the channel to perform identity authentication with the cloud server or device and establish a connection with them.
- 5) *Verification Table Theft Attack*: The attacker utilizes SQL injection to gain unauthorized access to the device registration information stored in the cloud server’s database. Then, they utilize this information to impersonate the cloud server and establish connections with devices.
- 6) *Stolen Device Attack*: Similar to the verification table theft attack, the attacker steals the registered devices of legitimate users and impersonates devices to communicate with the server.
- 7) *DoS Attack*: The attacker masquerades as a legal identity and floods the server with a large number of intercepted or constructed messages. It consumes computing and communication link resources, causing network paralysis.

D. Review of He et al.’s Scheme

He et al.’s scheme [36] involves two phases: 1) registration and 2) session key establishment for the D2C authentication.

- 1) *Registration*: Device ED_i inputs its identity Id_i and password Pw_i , computes $I_i = h(Id_i || Pw_i)$, and transmits I_i to the cloud server CS. Then, CS utilizes its private key X_{cs} , random value r_{cs} , identity Id_{cs} , and expiration time E_t to compute $Pid_i = h(I_i || Id_{cs} || r_{cs}) \oplus Id_{cs}$, $C_k = h(X_{cs} || Pid_i || E_t || r_{cs}) \oplus Id_{cs}$, $C'_k = C_k \oplus I_i$, $A_i = ENC_{X_{cs}}(Pid_i || E_t)$, $A'_i = A_i \oplus I_i$, $R_i = r_{cs} \oplus h(X_{cs} || Pid_i)$, $R'_i = R_i \oplus I_i$ and stores $\{R_i, E_t, Pid_i\}$ in its database. On the other side, ED_i stores $\{I_i, Pid_i, C'_k, A'_i, R'_i\}$.
- 2) *Authentication*: After registration, ED_i inputs its Id_i and Pw_i to compute I_i and C_k and verifies their trustworthiness. It produces the random value e_i and timestamp T_1 and calculates $A_i = A'_i \oplus I_i$, $R_i = R'_i \oplus I_i$, $N_i = ECC_{P_{cs}}(Pid_i || R_i)$ and $E_i = ENC_{C_k}(e_i, T_1, A_i)$, where P_{cs} is the public key of CS. ED_i transmits $\{N_i, T_1, E_i\}$ to CS, and CS verifies if they are legal from its database. Then, it gains $r_{cs} = R_i \oplus h(X_{cs} || Pid_i)$, $C_k = h(X_{cs} || Pid_i || E_t || r_{cs}) \oplus Id_{cs}$, E_i, A_i by decryption. It selects a random number q_i and produces $Q_i = h(A_i || C_k)$, $s_i = q_i \oplus C_k$, $w_i = h(Pid_i || e_i)$ and $T_i = ENC_{A_i}(s_i, w_i, T_2)$, where T_2 is the current timestamp. After receiving $\{T_i, T_2\}$ from CS, ED_i decrypts T_i and computes $q'_i = s_i \oplus C_k$ and $Q_i = h(A_i || C_k)$. Finally, it calculates the session key $sk_{ED_i} = h(e_i || C_k || Q_i || R_i || s_i)$. It sends the verification message and timestamp to CS and CS calculates the session key in the same way.

Herein, we discuss the weaknesses in He et al.’s protocol.

- 1) *Inability to Resist Clock Synchronization Attack*: In He et al.’s protocol, they utilize timestamps instead of nonce values to resist replay attacks. However, timestamps lead to the problem of clock synchronization in practical environments.

- 2) *Inability to Resist Stolen Device Attack*: In He's scheme, users store their privacy values $\{I_i, Pid_i, C'_k, A'_i, R'_i\}$ in devices. If the device is stolen by the adversary, he can effortlessly obtain these values. To start an attack, the attacker passes the verification of I_i, I_i^* and directly calculates $C_k = C'_k \oplus I_i$, $A_i = A'_i \oplus I_i$ and $R_i = R'_i \oplus I_i$. Given that the public key P_{CS} of the cloud server is known, the attacker can compute N_i and E_i by encryption. After obtaining $\{T_i, T_2\}$, the attacker can calculate the value of the session key.
- 3) *Inability to Resist Impersonation*: Based on the stolen attack, the attacker can impersonate a legitimate user because he already holds all the secrets of the stolen device.
- 4) *Inability to Resist Offline Password Guessing Attack*: The identity and password of a device are of low entropy, making them vulnerable to be cracked by the attacker within a limited dictionary space. This allows the attacker to obtain the correct I_i value.
- 5) *Computational Flaws*: He et al. calculated the computation cost as $11T_h + 2T_{en} + 3T_d + 2T_p$, where T_h, T_{en}, T_d, T_p refer to the time required for hash, encryption, decryption, and multiplication operations, respectively. In the scheme, the authors consider the cost of ECC encryption as the time required for a point multiplication operation. However, they are not equivalent in cryptographic systems. An ECC encryption typically involves multiple point addition and multiplication operations, and encryption that merely employs one point multiplication operation without any point addition is almost nonexistent or has extremely low security [40]. To address this problem, we measure the time required for a single classic ECC encryption and recalculate the communication overhead accordingly.

IV. PROPOSED SCHEME

In this section, we introduce the proposed scheme's registration and authentication in detail. The notations and mathematical definitions used in our work are listed in Table I. The device registration process is detailed in Table II, and the authentication details between D2C and D2D are shown in Tables III and IV, respectively.

A. Registration Phase

The registration phase for devices is listed as follows.

Step 1: $Dev_i \rightarrow CS : I_i$

The current device Dev_i inputs its identity id_i , password pw_i , and biometric f_i . It uses hash digest to process f_i and calculates $I_i = h(id_i || pw_i || h(f_i))$. It sends I_i to the cloud server through the secure channel.

Step 2: $CS \rightarrow Dev_i : \{B_i, D_i, E_i, V_i\}$

CS receives I_i , then it produces a random value r_i from Z_p^* , and calculates values as follows: $R_i = r_i P$, $B_i = h(I_i || R_i) x_{cs} \oplus r_i$, $C_i = h(x_{cs} || I_i)$, $D_i =$

TABLE I
NOTATIONS USED

Notation	Description
id_i	Identity of i th Device
pw_i	Password of i th Device
f_i	Biometric of i th Device User
Dev_i	i th Device
CS	Cloud Server
gw	Gateway
A	Adversary
P	Base Point of Elliptic Curve
Z_p^*	Multiplicative Group with Order $p - 1$
F_p	The Finite Field
$E_p(u, v)$	Elliptic Curve over F_p
P_{cs}	The Public Key of CS
x_{cs}	The Private Key of CS
sk	Session Key
r_i, m_1, m_2, n_i, n_j	Random Numbers
$Auth_1, Auth_2$	Verification Information in D2C
$Auth_i, Auth_j$	Verification Information in D2D
I_i	Hash Digest of i th User's Privacy
PID	Random Value Transmitted
B_i, D_i, E_i, V_i	Values Stored in i th Device
$h(\cdot)$	One-way Hash Algorithm
$\oplus$	XOR Operation of Bits
$\parallel$	Concatenation Operation
$\rightarrow$	Insecure Communication Channel
$\Rightarrow$	Secure Communication Channel

TABLE II
REGISTRATION PHASE

Device Dev_i	Cloud Server CS
input: id_i, pw_i, f_i $I_i = h(id_i pw_i h(f_i))$	
$\xRightarrow{I_i}$	produce: $r_i \in Z_p^*$ $R_i = r_i P$ $B_i = h(I_i R_i) x_{cs} \oplus r_i$ $C_i = h(x_{cs} I_i)$ $D_i = C_i \oplus I_i$ $E_i = C_i \oplus R_i$ $V_i = h(B_i D_i E_i I_i)$
	$\xleftarrow{B_i, D_i, E_i, V_i}$
stores: $\{B_i, D_i, E_i, V_i\}$	

$C_i \oplus I_i$, $E_i = C_i \oplus R_i$ and $V_1 = h(B_i || D_i || E_i || I_i)$, where x_{cs} is the private key of CS . V_i is used for verification in the authentication phases. Finally, CS returns $\{B_i, D_i, E_i, V_i\}$ to the current device. Dev_i stores them and ends the registration phase.

B. Authentication of D2C

After devices register in a secure channel environment, they gain permission to authenticate with each other and the cloud server. In this process, the channels are insecure, so messages transmitted in them may be obtained by attackers at any time. The specific processes of the authentication for D2C and D2D are defined as follows.

Step 1: $Dev_i \rightarrow CS : \{PID, M_1, Auth_1\}$

Dev_i inputs its password pw_i and biometric f_i , and computes $I_i = h(id_i || pw_i || h(f_i))$, $C_i = D_i \oplus I_i$,

TABLE III
AUTHENTICATION OF D2C

Device Dev_i	Cloud Server CS
input id_i, pw_i, f_i $I_i = h(id_i pw_i h(f_i))$ $C_i = D_i \oplus I_i$ $R_i = C_i \oplus E_i$ $V'_i = h(B_i D_i E_i I_i)$ verify: $V'_i \stackrel{?}{=} V_i$ produce: $m_1 \in Z_p^*$ $M_1 = m_1 P$ $PID = I_i \oplus m_1 P_{cs}$ $Auth_1 = h(I_i C_i M_1)$	$PID, M_1, Auth_1 \rightarrow$ $I_i = PID \oplus M_1 x_{cs}$ $C_i = h(x_{cs} I_i)$ $Auth'_1 = h(I_i C_i M_1)$ verify: $Auth'_1 \stackrel{?}{=} Auth_1$ produce: $m_2 \in Z_p^*$ $M_2 = m_2 P$ $sk = h(C_i I_i m_2 M_1)$ $Auth_2 = h(sk C_i I_i)$
$sk = h(C_i I_i m_1 M_2)$ $Auth'_2 = h(sk C_i I_i)$ verify: $Auth'_2 \stackrel{?}{=} Auth_2$	$M_2, Auth_2 \leftarrow$
Session Key: sk	

$R_i = C_i \oplus E_i$ and verification message $V'_i = h(B_i || D_i || E_i || I_i)$. Then, it checks if V'_i is equal to V_i . If verification succeeds, it produces a random number m_1 from Z_p^* , and calculates $M_1 = m_1 P$, $PID = I_i \oplus m_1 P_{cs}$. Finally, Dev_i produces $Auth_1 = h(I_i || C_i || M_1)$ and transmits $\{PID, M_1, Auth_1\}$ to CS .

Step 2: $CS \rightarrow Dev_i : \{M_2, Auth_2\}$

CS recovers $I_i = PID \oplus M_1 x_{cs}$ according to PID and its private key x_{cs} , and calculates $C_i = h(x_{cs} || I_i)$, $Auth'_1 = h(I_i || C_i || M_1)$. After verifies $Auth'_1$ is equal to $Auth_1$, it produces m_2 from Z_p^* and $M_2 = m_2 P$. It computes the session key $sk = h(C_i || I_i || m_2 M_1)$ and the verification message $Auth_2 = h(sk || C_i || I_i)$. Then, CS transmits $\{M_2, Auth_2\}$ to Dev_i .

Step 3: Generation of the Session Key sk

Dev_i produces $sk = h(C_i || I_i || m_1 M_2)$, and verifies if $Auth'_2 = h(sk || C_i || I_i)$ is equal to $Auth_2$.

Device Dev_i and CS complete the current round of authentication and key negotiation after the end of the above steps. With the secure session key sk , they can update parameters at the next stage, such as R_i and other random values to ensure higher security.

C. Authentication of D2D

Step 1: $Dev_i \rightarrow Dev_j : \{R_i, N_i, T_i, Auth_i\}$

Dev_i inputs its password pw_i and biometric f_i , and computes $I_i = h(id_i || pw_i || h(f_i))$, $C_i = D_i \oplus I_i$, $R_i = C_i \oplus E_i$, and $T_i = h(I_i || R_i)$. It produces n_i from

TABLE IV
AUTHENTICATION OF D2D

Device Dev_i	Gateway gw	Device Dev_j
input id_i, pw_i, f_i $I_i = h(id_i pw_i h(f_i))$ $C_i = D_i \oplus I_i$ $R_i = C_i \oplus E_i$ $T_i = h(I_i R_i)$ produce: $n_i \in Z_p^*$ $N_i = n_i P$ $Auth_i = n_i \oplus B_i$	$R_i, N_i, T_i, Auth_i \rightarrow$ input id_j, pw_j, f_j $I_j = h(id_j pw_j h(f_j))$ $C_j = D_j \oplus I_j$ $R_j = C_j \oplus E_j$ $Auth_i P \stackrel{?}{=} T_i P_{cs} \oplus R_i \oplus N_i$ produce: $n_j \in Z_p^*$ $N_j = n_j P$ $T_j = h(I_j R_j)$ $Auth_j = n_j \oplus B_j$ $sk = h(T_i T_j N_i n_j)$	$R_j, N_j, T_j, Auth_j \leftarrow$ $Auth_j P \stackrel{?}{=} T_j P_{cs} \oplus R_j \oplus N_j$ $sk = h(T_i T_j N_i n_j)$
Session Key: sk		

Z_p^* , and calculates $N_i = n_i P$. Finally, Dev_i produces $Auth_i = n_i \oplus B_i$ and transmits $\{R_i, N_i, T_i, Auth_i\}$ to Dev_j .

Step 2: $Dev_j \rightarrow Dev_i : \{R_j, N_j, T_j, Auth_j\}$

Similar to Dev_i , Dev_j recovers I_j , C_j , R_j from $I_j = h(id_j || pw_j || h(f_j))$, $C_j = D_j \oplus I_j$, $R_j = C_j \oplus E_j$, and $T_j = h(I_j || R_j)$. It verifies the validity of Dev_i by checking if $Auth_i P$ is equal to $T_i P_{cs} \oplus N_i \oplus R_i$. This is because if $Auth_i$ is legal then we have $Auth_i P = (n_i \oplus B_i) P = (n_i \oplus T_i x_{cs} \oplus r_i) P = N_i \oplus T_i P_{cs} \oplus R_i$. After verification successes, Dev_j moves to the following steps: it produces random value $n_j \in Z_p^*$, and computes $N_j = n_j P$, $T_j = h(I_j || R_j)$ and $Auth_j = n_j \oplus B_j$. Finally, Dev_j sends $\{R_j, N_j, T_j, Auth_j\}$ to Dev_i and the session key between both devices is $sk = h(T_i || T_j || N_i n_j)$.

Step 3: Generation of Session Key sk

Dev_i receives $\{R_j, N_j, T_j, Auth_j\}$, and verifies if $Auth_j P$ is equal to $N_j \oplus T_j P_{cs} \oplus R_j$. The session key in the side of Dev_i is calculated as $sk = h(T_i || T_j || N_i n_j)$.

Devices Dev_i and Dev_j complete the current round of authentication and key negotiation after the end of the above steps.

V. SECURITY ANALYSIS

A. Informal Security Analysis

1) *Mutual Authentication*: Our proposed scheme contains information used for verification during transmission. In the authentication between devices and the cloud server,

device Dev_i produces $Auth_1 = h(I_i || C_i || M_1)$ and $Auth_i = n_i \oplus B_i$. Only if Dev_i inputs its identity, password, and biometric can it get valid $I_i = h(id_i || pw_i || h(f_i))$, and compute R_i, C_i, B_i for $Auth_1, Auth_i$. On the other side, in the D2C authentication phase, the cloud server provides its legal identity by returning I_i and C_i in $Auth_2$, which is computed by its private key x_{cs} . In the D2D authentication, devices verify each other through R_i and R_j in $Auth_i, Auth_j$. Therefore, devices and the cloud server complete the process of mutual authentication.

- 2) *Share of Session Keys*: In the D2C authentication, Dev_i and CS share the session key by calculating $sk = h(C_i || I_i || m_1 M_2)$ and $sk = h(C_i || I_i || M_1 m_2)$. And Dev_i and Dev_j share the session key $sk = h(T_i || T_j || n_i n_j)$ and $sk = h(T_i || T_j || N_i n_j)$. Therefore, our protocol supports sharing secure session keys.
- 3) *Device Anonymity*: Devices hide their identity by computing $I_{id} = h(id || pw_{id} || h(f_{id}))$. In the actual communication process, unless the attacker breaks the trap door of the hash function, it is impossible for him to obtain the device's identity. Thus, the scheme provides device anonymity and untraceability.
- 4) *Stolen Device Attack*: In this scheme, the parameters stored in devices are $\{B_i, D_i, E_i, V_i\}$. It can be seen from the registration and authentication stages that if the attacker merely knows the information transmitted in the device and public communication channel but does not obtain pw_i and $h(f_i)$, he is unable to pass the authentication of the other communicating party. Through this approach, we defend against the stolen device attack successfully.
- 5) *Offline Password Guessing Attack*: The offline password guessing attack refers to the process in which attackers attempt to obtain passwords without accessing the user's system, such as by cracking hash values. Therefore, it is possible for an attacker to gain id and password pw successfully. However, even if he gets these values, he is required to obtain the user's $h(f_i)$ information at the same time for launching intrusion, which is impossible to achieve.
- 6) *Man-in-the-Middle Attack*: In the above description, we draw the conclusion that the scheme can complete mutual authentication. Thus, it resists the man-in-the-middle attack.
- 7) *Privileged Insider Attack*: A privileged attacker may get information in a secure channel. That is, it intercepts data transferred between the device and the cloud server during the registration phase. In this protocol, the data packets transmitted through the secure channel contain $\{I_i, B_i, D_i, E_i, V_i\}$. I_i is protected by the hash algorithm. Therefore, the privileged adversary cannot guess the user's pw_i and $h(f_i)$ through them.
- 8) *Stolen-Verifier Attack*: In our scheme, the cloud server side does not store any private data about the device either during the registration or authentication phase, thereby preventing stolen-verifier attacks.
- 9) *Impersonation Attack*: The attacker attempts to masquerade as a legal cloud server and device in an

impersonation attack. In the D2C authentication, he may generate random value m'_1 and false data I'_i, C'_i , and produce data $\{PID', M'_1, Auth'_1\}$, where the values of them are calculated as $M'_1 = m'_1 P$, $PID' = I' \oplus m'_1 P_{cs}$, $Auth'_1 = h(I'_i || C'_i)$. He cannot pass verification as C_i is calculated as $C_i = I_i \oplus D_i$ on the device side. Without valid I_i , the attacker cannot get C_i and masquerade as Dev_i . On the other side, if an attacker tries to impersonate CS, he demands legal value $Auth_2$, which is computed as $Auth_2 = h(sk || C_i || I_i)$. Without CS's private key x_{cs} , he cannot gain the value of I_i , and the attack fails. The D2D phase resists impersonation attacks in the same way. Both participants need to provide accurate $Auth_{id}$ to satisfy $Auth_{id} P = T_{id} P_{cs} \oplus R_{id} \oplus N_{id}$. Only devices registered on the cloud server side hold legal B_{id} to build $Auth_{id}$, and fake users cannot be authenticated successfully.

- 10) *Replay Attack*: Suppose an attacker performs a replay attack by intercepting and resending information. Due to the random numbers, such as m_1, m_2, n_i, n_j , all messages transmitted in the channel in each session are different. Thus, the D2C and D2D authentication phases resist the replay attack successfully.
- 11) *Clock Synchronization*: In our scheme, we employ random numbers instead of timestamps to defend against the clock synchronization problem.
- 12) *DoS Attack*: In our protocol, there is a verification message after each communication transmission. If the authentication fails, the verifier will terminate the communication. If an attacker keeps sending incorrect messages that cause authentication to fail, he will be rejected from communicating, thereby resisting the DoS attack.
- 13) *Perfect Forward Security*: Forward security means that once the server key is compromised, it does not affect the encrypted data in the past. That is, an attacker cannot compute previous session keys even if he has obtained the key x_{cs} . Forward security is an essential feature of an authentication protocol or system because it prevents key leakage from negatively affecting the long-term security of encrypted information. In our proposed protocol, the confidentiality of sk is based on the ECDHP. Random values m_1, m_2, n_i, n_j cannot be gained by attackers. Even if the attacker obtains the value of the cloud server's private key x_{cs} , it is almost impossible for him to calculate the specific value of m_1, m_2, n_i, n_j through M_1, M_2, N_i, N_j with the elliptic discrete logarithm problem. Therefore, our scheme provides perfect forward security.

B. Formal Security Analysis

We use the password-based authenticated key exchange security model ROR for both parties proposed by Abdalla et al. [41]. The model is a widely used method in protocol design and can prove the theoretical security of session keys. In the ROR security model, an attacker can choose which message to send, which can be either real

or random, rather than a predetermined challenge message. This model is often used to assess the security of encryption algorithms and protocols in real-world attack scenarios. To meet the requirements of the ROR security model, encryption algorithms or protocols must be able to prevent an attacker from inferring any information about the original message from the cryptographic text. In addition, algorithms or protocols must be able to provide confidentiality, integrity, and nonrepudiation to protect information in communication from tampering or forgery. We set the following entities in the model: the device Dev_{id} participating in the protocol with its identity id , adversary A , and the cloud server CS .

Take the D2D authentication as an instance: A can use random oracles (ROs) to produce a bounded number of queries and apply them to simulate actual attacks. The ability of A brought by RO is formally defined by the following queries.

- 1) *Execute*(Dev_i, Dev_j): The query launches the eavesdropping attack based on the request of adversary A , and the detailed session progress of user Dev_i and Dev_j is monitored by A . The query returns messages transmitted between Dev_i and Dev_j in the public channel.
- 2) *Send*(Dev_i, Dev_j, m): The query launches active attacks based on the request of adversary A , such as replay and impersonation attacks. A transmits a modified message m to Dev_i or Dev_j , and they generate corresponding information as a reply. Then, the oracle returns the running result.
- 3) *Corrupt*(Dev_i, μ): This query allows an attacker to obtain parameter information that is stored on the device for a long time, such as I_i . In the protocol environment, we divide the query into three scenarios: if $\mu = 0$, it returns the parameters $\{B_i, D_i, E_i, V_i\}$ stored in Dev_i ; if $\mu = 1$, it returns pw_i of Dev_i ; if $\mu = 2$, it returns f_i of Dev_i .
- 4) *Reveal*(Dev_i): The query simulates the reveal of session keys. If the adversary has gained sk , this query returns it.
- 5) *Hash*(m): *Hash*(m) represents the hash query. A launches an active attack in this query by generating a hash digest to Dev_i .
- 6) *Test*(Dev_i, Dev_j): The purpose of the query is to test how much privacy adversary A can master to obtain the secret key from Dev_i and Dev_j and express the semantic security of the session key. The final results are as follows: if the session key is not obtained during the test process, end the query. Otherwise, A starts a game of tossing the coin. Suppose the result value of tossing is the bit c , where $c = 0$ or $c = 1$. The output of this toss results in the return value of the *Test*(Dev_i, Dev_j) query. If the tossing result is positive, the query sets $c = 1$, and the oracle returns the initial session key. Otherwise, the query sets $c = 0$, and the oracle returns a random value with the same scale as the session key. In addition, A can appeal to confirm whether the value given by this query is true or false. It is additionally noted that no matter how many times A executes the above queries, the *Test*(Dev_i, Dev_j) process must be executed finally.

Furthermore, this query can only be executed in a new session and only once.

Based on the above queries, we introduce the definition of $Adv_{\Pi}^{ake}(A)$ as follows.

Definition 3: $Adv_{\Pi}^{ake}(A)$ denotes the advantage of adversary A to break the current protocol, and it represents the success possibility of A . $Adv_{\Pi}^{ake}(A)$ is computed as

$$Adv_{\Pi}^{ake}(A) = 2 \cdot pr[c = c'] - 1 \quad (2)$$

where c' is the value obtained by A . If c' is equal to c , then A wins, thus it can also be calculated as

$$Adv_{\Pi}^{ake}(A) = 2 \cdot pr[SUCC] - 1 \quad (3)$$

where $SUCC$ represents the success position of A .

Definition 4: A protocol Π is called semantically secure if $Adv_{\Pi}^{ake}(A)$ is negligible.

Here, we get Theorem 1 to prove the semantic security of the protocol based on the mentioned security model.

Theorem 1: A is a probabilistic polynomial time (PPT) adversary against the semantically secure of the proposed scheme Π in D2D authentication, then we have

$$Adv_{\Pi}^{ake}(A) \leq \frac{q_h^2}{2^{l_h}} + \frac{q_s}{2^{l_h}} + \frac{(q_s + q_e)^2}{2^{l_h+1}} + \max\left(\frac{1}{|pw|}, \frac{1}{2^{l_b}}\right) \cdot q_s + q_h Adv_A^{ECDHP} \quad (4)$$

where $|pw|$ represents the dictionary size of the user's password. q_s , q_e , and q_h represent the times of *Send*, *Execute*, and *Hash* queries, respectively. Adv_A^{ECDHP} is the possibility of solving the ECDHP problem for A . l_h and l_b are the sizes of hash strings and biometrics.

From (4), we conclude that the protocol is semantically secure, and it is proved as follows.

Proof: We define the following security games called: Gm_0 , Gm_1 , Gm_2 , Gm_3 , Gm_4 , and Gm_5 to prove that the proposed work is secure against A . We introduce a definition of event $SUCC_i$, which indicates the success possibility of the adversary A winning each game Gm_i through the queries introduced above.

Gm_0 : In the game where Gm_0 simulates the actual attack, A does not execute any queries. If A succeeds, it means that it passes the test query and obtains a coin result of $c = 1$, that is

$$Adv_{\Pi}^{ake}(A) = |pr[SUCC_0] - \frac{1}{2}|. \quad (5)$$

Gm_1 : In Gm_1 , player A attempts to launch an eavesdropping attack on the protocol by using the *Execute*(Dev_i, Dev_j) query. The final session key is produced by $\{T_i, T_j, n_i, n_j\}$. Through eavesdropping attacks A cannot gain these values, and the probability of A succeeding in this round is the same as that in the previous game Gm_0 , then we have

$$pr[SUCC_0] = pr[SUCC_1]. \quad (6)$$

Gm_2 : Based on Gm_1 , player A attempts to launch active attacks by applying *Send*(Dev_i, Dev_j, m) and *Hash*(m) queries. However, all messages transmitted over insecure channels

TABLE V
SECURITY COMPARISONS

Security features	[23]	[24]	[26]	[28]	[36]	[30]	[31]	[32]	[35]	Our Scheme
Mutual authentication	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓
Share of session keys	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Device anonymity	✗	✓	✓	✓	✓	✗	✗	✓	✗	✓
Stolen device attack	✓	✓	✗	✗	✗	✓	✓	✓	✓	✓
Off-line password guessing attack	✓	✓	✓	✗	✗	✓	✓	✓	✓	✓
Man-in-the-middle Attack	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Impersonation attack	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓
Replay attack	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Clock synchronization	✓	✓	✓	✗	✗	✓	✗	✗	✓	✓
Stolen-verifier attack	✓	✗	✗	✗	✓	✗	✓	✗	✓	✓
Perfect forward security	✓	✓	✓	✗	✓	✓	✓	✓	✓	✓

contain fresh and dynamic random numbers, and A cannot successfully find hash collisions. Based on the birthday paradox and difference lemma, the probability of a hash collision can be defined as $(q_h^2/2^{l_h+1})$. Furthermore, we use two random numbers, n_i and n_j , in the insecure channel transmission, and the probability of their transcripts colliding is $((q_s + q_e)^2/2^{l_h+1})$. By combining these probabilities, we have

$$|pr[SUCC_2] - pr[SUCC_1]| \leq \frac{q_h^2 + (q_s + q_e)^2}{2^{l_h+1}}. \quad (7)$$

Gm_3 : Gm_3 follows the same process as Gm_2 , which mainly considers the additional case that A finds a collision without sending any hash query, that is

$$|pr[SUCC_3] - pr[SUCC_2]| \leq \frac{q_s}{2^{l_h+1}}. \quad (8)$$

Gm_4 : In Gm_4 , the attacker launches the corrupt query $Corrupt(Dev_i, \mu)$ on the basis of Gm_3 to obtain data that stored in Dev_i . In this query, it is assumed that A holds the possibility of obtaining the information stored in devices. The probability of success for A in this round refers to its probability of gaining pw_i and f_i . To acquire pw_i , A launches $Corrupt(Dev_i, 0)$ and $Corrupt(Dev_i, 1)$, and the success probability is $(q_s/|pw|)$. For f_i , A calls for $Corrupt(Dev_i, 0)$ and $Corrupt(Dev_i, 2)$ and the possibility becomes $(q_s/2^{l_b})$. Herein, we can calculate

$$|pr[SUCC_4] - pr[SUCC_3]| \leq q_s \cdot \max\left(\frac{1}{|pw|}, \frac{1}{2^{l_b}}\right). \quad (9)$$

Gm_5 : In Gm_5 , the attacker attempts to guess the value of sk . If A solves the ECDHP with the success possibility Adv_A^{ECDHP} , he can obtain $\{N_i, N_j, T_i, T_j\}$ by eavesdropping, and therefore calculates $\{n_i, n_j\}$. Even if A overcomes the ECDHP, he cannot compute sk directly and needs to launch a success hash oracle. Thus, the attack advantage of A becomes

$$|pr[SUCC_5] - pr[SUCC_4]| \leq q_h Adv_A^{ECDHP} + \frac{q_h^2}{2^{l_h+1}}. \quad (10)$$

After A ends the game Gm_5 , he has no choice and can only launch $Test(Dev_i, Dev_j)$ query to guess the value of c , so we have

$$|pr[SUCC_5]| = \frac{1}{2}. \quad (11)$$

TABLE VI
PARAMETERS USED

Notation	Description	Required Time
$T_{ }$	concatenation operation	-
$T_{\oplus}$	XOR operation	-
T_{pm}	point multiplication	0.33 ms
T_a	point addition	0.037 ms
T_h	hash operation	0.048 ms
T_{sym}	symmetric encryption/decryption	0.022 ms
T_{pb}	bilinear pairing operation	1.7 ms
T_{ee}	ECC encryption operation	1.2 ms
T_{ed}	ECC decryption operation	0.83 ms

From (6)–(11), it can be proved that

$$\begin{aligned} Adv_{\Pi}^{ake}(A) &= |pr[SUCC_0] - \frac{1}{2}| \\ &= |pr[SUCC_1] - pr[SUCC_5]| \\ &\leq \frac{q_h^2}{2^{l_h}} + \frac{q_s}{2^{l_h+1}} + \frac{(q_s + q_e)^2}{2^{l_h+1}} \\ &\quad + \max\left(\frac{1}{|pw|}, \frac{1}{2^{l_b}}\right) \cdot q_s + q_h Adv_A^{ECDHP}. \end{aligned} \quad (12)$$

From the above description, (4) in Theorem 1 can be proved, and the D2D authentication is semantically secure. Similarly, it can be proven that the D2C authentication is semantically secure. ■

VI. COMPARISONS AND SIMULATIONS

A. Security Comparisons

We provide a security comparison between our proposed protocol and previous research in Table V. Previous studies demonstrate weaknesses in terms of security, such as the stolen-verifier attack, replay attack, man-in-the-middle attack, and so on. On the basis of these works, we make improvements to address these shortcomings and fulfill all security requirements. Additionally, our protocol provides perfect forward security for D2D and D2C authentication while reducing storage costs.

B. Computation Consumption

As presented in Table VI, we utilize the Ubuntu 20.04 system with an i7-3.10Hz processor and 8.0-GB RAM

TABLE VII
COMPUTATION AND COMMUNICATION CONSUMPTION
COMPARISONS (D2C)

Schemes	Required Time	Results	Cost
[23]	$8T_h + 8T_{pm}$	3.024 ms	1760 bits
[24]	$8T_h + 7T_{pm}$	2.694 ms	1760 bits
[26]	$9T_h + 10T_{pm}$	3.372 ms	1760 bits
[28]	$10T_h + 6T_{pm} + 2T_a$	2.534 ms	2048 bits
[36]	$11T_h + 5T_{sym} + T_{ee} + T_{ed}$	2.668 ms	892 bits
Ours	$10T_h + 6T_{pm}$	2.46 ms	1248 bits

TABLE VIII
COMPUTATION AND COMMUNICATION CONSUMPTION
COMPARISONS (D2D)

Schemes	Required Time	Results	Cost
[30]	$2T_h + 4T_{pm} + 2T_{pb} + 2T_a$	4.36 ms	1280 bits
[31]	$6T_h + 8T_{pm} + 2T_a$	3.002 ms	1504 bits
[32]	$14T_h + 8T_{pm}$	3.312 ms	1344 bits
[35]	$12T_h + 14T_{pm} + 4T_a$	5.344 ms	3200 bits
Ours	$8T_h + 6T_{pm}$	2.364 ms	2176 bits

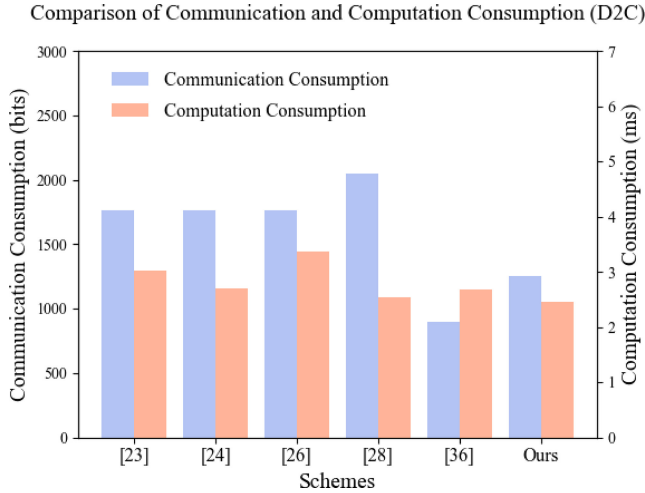


Fig. 2. Comparison of communication and computation consumption (D2C).

to conduct the necessary computation operations. The table displays the operations represented by each time parameter. In practical scenarios, the time required for concatenation ($T_{||}$) and XOR operation ($T_{\oplus}$) can be negligible. Our simulation accumulates the time required for symmetric encryption (T_{sym}), the time required for hash operations (T_h), the time for point multiplication and addition on the elliptic curve (T_{pm} , T_a), the time for bilinear pairing (T_{pb}), and time required for ECC encryption and decryption operation (T_{ee} , T_{ed}). Based on the above conclusion, we calculate the computation time and communication cost of prior works and our scheme as shown in Tables VII and VIII and Figs. 2 and 3.

C. Communication Cost

Without losing generality, we define the bit lengths for various parameters in our protocol as follows: the output of the hash digest, random number, device identity, timestamp, and ECC point have bit lengths of 160, 160, 160, 32, and

Comparison of Communication and Computation Consumption (D2D)

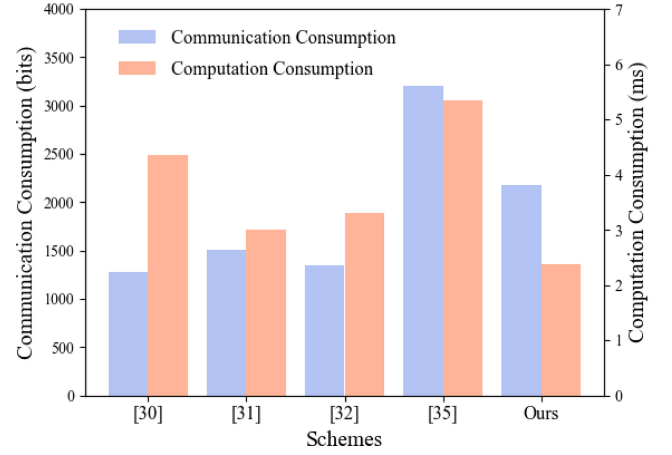


Fig. 3. Comparison of communication and computation consumption (D2D).

320 bits, respectively. The results of communication cost are shown in Tables VII and VIII and Figs. 2 and 3. Our D2C and D2D solutions hold time delay with 2.46 and 2.364 ms and communication cost with 1248 and 2176 bits in per interaction, respectively. It is evident from simulation results that our work achieves security objectives while surpassing the performance of most protocols.

VII. CONCLUSION

This work proposes an ACLAP for device authentication in the IoT. We conduct a thorough analysis of security issues in device-server communication networks and the requirements for different applications. Our proposed method avoids security vulnerabilities and improves the existing solution proposed by He et al. [36]. By utilizing the ECC and hash operations, our scheme eliminates the need for storing devices' privacy on the server and reduces communication and computation consumption. It mitigates the risk of verifier table theft, enhancing the overall security of the authentication. In addition, founded on the device-server communication protocol, we propose an anonymous communication scheme between devices. This expansion increases the practical applicability of our scheme in various IoT environments, including vehicular networks, smart cities, and medical facilities.

For future exploration directions, we intend to extend our solution to cover more scenarios. Our current scheme is designed for a two-party model and does not consider authentication scenarios involving multiple entities. Moreover, there are numerous authentication technologies that can be integrated with widespread applications of hardware devices in the IoT, such as PUF and RFID techniques. In the future, we plan to extend our work to communication scenarios involving more participating roles and various authentication technologies.

REFERENCES

- [1] H. Zhu et al., "Smart healthcare in the era of Internet-of-Things," *IEEE Consum. Electron. Mag.*, vol. 8, no. 5, pp. 26–30, Sep. 2019.

- [2] S. L. Ullo and G. R. Sinha, "Advances in smart environment monitoring systems using IoT and sensors," *Sensors*, vol. 20, no. 11, p. 3113, 2020.
- [3] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "IoT: Internet of Threats? A survey of practical security vulnerabilities in real IoT devices," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8182–8201, Oct. 2019.
- [4] M. B. M. Noor and W. H. Hassan, "Current research on Internet of Things (IoT) security: A survey," *Comput. Netw.*, vol. 148, pp. 283–294, Jan. 2019.
- [5] M. A. Ferrag, L. A. Maglaras, H. Janicke, J. Jiang, and L. Shu, "Authentication protocols for Internet of Things: A comprehensive survey," *Security Commun. Netw.*, vol. 2017, Nov. 2017, Art. no. 6562953.
- [6] O. Yousuf and R. N. Mir, "A survey on the Internet of Things security: State-of-art, architecture, issues and countermeasures," *Inf. & Comput. Secur.*, vol. 27, no. 2, pp. 292–323, 2019.
- [7] H. Haddadpajouh, A. Dehghantanha, R. M. Parizi, M. Aledhari, and H. Karimipour, "A survey on Internet of Things security: Requirements, challenges, and solutions," *Internet Things*, vol. 14, Jun. 2021, Art. no. 100129.
- [8] R. Yugha and S. Chithra, "A survey on technologies and security protocols: Reference for future generation IoT," *J. Netw. Comput. Appl.*, vol. 169, Nov. 2020, Art. no. 102763.
- [9] X. Li, J. Peng, M. S. Obaidat, F. Wu, M. K. Khan, and C. Chen, "A secure three-factor user authentication protocol with forward secrecy for wireless medical sensor network systems," *IEEE Syst. J.*, vol. 14, no. 1, pp. 39–50, Mar. 2019.
- [10] B. D. Deebak and F. Al-Turjman, "Smart mutual authentication protocol for cloud based medical healthcare systems using Internet of Medical Things," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 2, pp. 346–360, Feb. 2020.
- [11] X. Liu, Z. Guo, J. Ma, and Y. Song, "A secure authentication scheme for wireless sensor networks based on DAC and Intel SGX," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3533–3547, Mar. 2021.
- [12] P. Gope, A. K. Das, N. Kumar, and Y. Cheng, "Lightweight and physically secure anonymous mutual authentication protocol for real-time data access in industrial wireless sensor networks," *IEEE Trans. Ind. Inf.*, vol. 15, no. 9, pp. 4957–4968, Sep. 2019.
- [13] S. Son, J. Lee, Y. Park, Y. Park, and A. K. Das, "Design of blockchain-based lightweight V2I handover authentication protocol for VANET," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 3, pp. 1346–1358, May 2022.
- [14] P. Wang and Y. Liu, "SEMA: Secure and efficient message authentication protocol for VANETs," *IEEE Syst. J.*, vol. 15, no. 1, pp. 846–855, Mar. 2021.
- [15] M. Ebrahimabadi, M. Younis, and N. Karimi, "A PUF-based modeling-attack resilient authentication protocol for IoT devices," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3684–3703, Mar. 2021.
- [16] P. Gope and B. Sikdar, "Lightweight and privacy-preserving two-factor authentication scheme for IoT devices," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 580–589, Feb. 2019.
- [17] F. Farha, H. Ning, K. Ali, L. Chen, and C. Nugent, "SRAM-PUF-based entities authentication scheme for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5904–5913, Apr. 2020.
- [18] M. Ghahramani, R. Javidan, and M. Shojafar, "A secure biometric-based authentication protocol for global mobility networks in smart cities," *J. Supercomput.*, vol. 76, pp. 8729–8755, Jan. 2020.
- [19] Y. Zhang, B. Li, B. Liu, Y. Hu, and H. Zheng, "A privacy-aware PUFs-based multiserver authentication protocol in cloud-edge IoT systems using blockchain," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 13958–13974, Sep. 2021.
- [20] A. K. Maurya and V. N. Sastry, "Fuzzy extractor and elliptic curve based efficient user authentication protocol for wireless sensor networks and Internet of Things," *Information*, vol. 8, no. 4, p. 136, 2017.
- [21] A. A. Alamr, F. Kausar, J. Kim, and C. Seo, "A secure ECC-based RFID mutual authentication protocol for Internet of Things," *J. Supercomput.*, vol. 74, pp. 4281–4294, Sep. 2018.
- [22] L. Harn and J. Ren, "Generalized digital certificate for user authentication and key establishment for secure communications," *IEEE Trans. Wireless Commun.*, vol. 10, no. 7, pp. 2372–2379, Jul. 2011.
- [23] K.-H. Wang, C.-M. Chen, W. Fang, and T.-Y. Wu, "A secure authentication scheme for Internet of Things," *Pervasive Mobile Comput.*, vol. 42, pp. 15–26, Dec. 2017.
- [24] S. Kumari, M. Karupiah, A. K. Das, X. Li, F. Wu, and N. Kumar, "A secure authentication scheme based on elliptic curve cryptography for IoT and cloud servers," *J. Supercomput.*, vol. 74, pp. 6428–6453, Apr. 2018.
- [25] R. Melki, H. N. Noura, and A. Chehab, "Lightweight multi-factor mutual authentication protocol for IoT devices," *Int. J. Inf. Security*, vol. 19, pp. 679–694, Dec. 2020.
- [26] P. K. Panda and S. Chattopadhyay, "A secure mutual authentication protocol for IoT environment," *J. Reliable Intell. Environ.*, vol. 6, pp. 79–94, Jan. 2020.
- [27] M. Pan, D. He, X. Li, S. Chan, E. Panaousis, and Y. Gao, "A lightweight certificateless non-interactive authentication and key exchange protocol for IoT environments," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, 2021, pp. 1–7.
- [28] X. Ding, X. Wang, Y. Xie, and F. Li, "A lightweight anonymous authentication protocol for resource-constrained devices in Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 3, pp. 1818–1829, Feb. 2021.
- [29] Z. Shang, M. Ma, and X. Li, "A secure group-oriented device-to-device authentication protocol for 5G wireless networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 11, pp. 7021–7032, Nov. 2020.
- [30] Q. Li, C.-F. Hsu, K.-K. Raymond Choo, and D. He, "A provably secure and lightweight identity-based two-party authenticated key agreement protocol for vehicular ad hoc networks," *Security Commun. Netw.*, vol. 2019, Dec. 2019, Art. no. 7871067.
- [31] B. A. Alzahrani, S. A. Chaudhry, A. Barnawi, A. Al-Barakati, and T. Shon, "An anonymous device to device authentication protocol using ECC and self certified public keys usable in Internet of Things based autonomous devices," *Electronics*, vol. 9, no. 3, p. 520, 2020.
- [32] R. Hajian, A. Haghighat, and S. H. Erfani, "A secure anonymous D2D mutual authentication and key agreement protocol for IoT," *Internet Things*, vol. 18, May 2022, Art. no. 100493.
- [33] B. Seok, J. C. S. Sicato, T. Erzhen, C. Xuan, Y. Pan, and J. H. Park, "Secure D2D communication for 5G IoT network based on lightweight cryptography," *Appl. Sci.*, vol. 10, no. 1, p. 217, 2019.
- [34] D. Fang, Y. Qian, and R. Q. Hu, "A flexible and efficient authentication and secure data transmission scheme for IoT applications," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3474–3484, Apr. 2020.
- [35] S. Li, T. Zhang, B. Yu, and K. He, "A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT," *IEEE Sensors J.*, vol. 21, no. 4, pp. 5487–5501, Feb. 2021.
- [36] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, "A lightweight authentication and key exchange protocol with anonymity for IoT," *IEEE Trans. Wireless Commun.*, vol. 22, no. 11, pp. 7862–7872, Nov. 2023.
- [37] N. Kobitz, "Elliptic curve cryptography," *Math. Comput.*, vol. 48, no. 117, pp. 203–209, 1987.
- [38] Y. Chen et al., "ECC-based authenticated key agreement protocol for industrial control system," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 4688–4697, Mar. 2022.
- [39] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, Mar. 1983.
- [40] R. Bhanot and R. Hans, "A review and comparative analysis of various encryption algorithms," *Int. J. Security Appl.*, vol. 9, no. 4, pp. 289–306, 2015.
- [41] M. Abdalla, P.-A. Fouque, and D. Pointcheval, "Password-based authenticated key exchange in the three-party setting," in *Proc. Int. Workshop Public Key Cryptography*, 2005, pp. 65–84.



Shanyao Ren received the B.S. degree from Shen Yuan Honors College, Beihang University, Beijing, China, in 2021, where she is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology.

Her research interests include cryptography and security of aerospace information network.



Yizhong Liu (Member, IEEE) received the B.S. degree in electronic and information engineering and the Ph.D. degree from Beihang University, Beijing, China, in 2014 and 2021, respectively.

He is currently an Assistant Professor with the School of Cyber Science and Technology, Beihang University. He has authored over 20 publications, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, CSR, COSE, COMNET, FGCS, ACISP, and TrustCom. His research interests include information security, cryptography, and blockchain.



Jianwei Liu (Senior Member, IEEE) received the B.S. and M.S. degrees in electronic engineering from Shandong University, Jinan, China, in 1985 and 1988, respectively, and the Ph.D. degree in communication engineering from Xidian University, Xi'an, China, in 1998.

He is currently a Professor and the Dean of the School of Cyber Science and Technology, Beihang University, Beijing, China. His current research interests include cryptographic protocol design, security on wireless and mobile network, computer network security, and cryptography. He has published six books and more than 200 papers in his research fields.

Prof. Liu has been awarded the First Prize of Technological Invention of China. He is a Senior Member of the Chinese Institute of Electronics, the Director of the Chinese Association for Cryptologic Research, and a member of Teaching Steering Committee of Information Security of MOE, China.



Bei yuan Yu received the B.S. degree from Northeastern University, Shenyang, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology, Beihang University, Beijing, China.

His research interest includes network application security.



Dongyu Li (Member, IEEE) received the B.S. and Ph.D. degrees in control science and engineering from Harbin Institute of Technology, Harbin, China, in 2016 and 2020, respectively.

He was a joint Ph.D. student with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore, from 2017 to 2019, where he was a Research Fellow with the Department of Biomedical Engineering from 2019 to 2021. He is currently an Associate Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China, where he is also a Research Scientist with Tianmushan Laboratory. His research interests include networked system cooperation, adaptive systems, and spacecraft engineering.